Structural (White-Box) Testing

Mata Kuliah:
Pengujian Perangkat Lunak (CIF62128)
Disusun oleh:
Software Engineering Lab.

Version: 1.0
Date: 11; Month: February; Year: 2026
© Fakultas Ilmu Komputer, Universitas Brawijaya

filkom.ub.ac.id

Agenda

- Dasar Structural (White-Box) Testing
- Path Analysis & Basis Path Testing
- Coverage Criteria dalam White-Box Testing
- Code Coverage Tools
- Tanya Jawab & Kesimpulan

© Fakultas Ilmu Komputer, Universitas Brawijaya

Apa itu White-Box Testing?

- **Definisi**: Teknik pengujian perangkat lunak di mana **struktur internal, desain, dan implementasi** dari suatu komponen diuji secara langsung (Ammann & Offutt, 2016).

- **Fokus**: Berbeda dengan Black-Box yang berfokus pada fungsionalitas (I/O), White-Box berfokus pada aliran data dan kontrol program (Control Flow).

- **Kapan dilakukan?** Biasanya pada level Unit Testing dan Integration Testing oleh Software Engineer atau SDET (Software Development Engineer in Test).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Unit Testing

- Unit testing focuses verification effort on **the smallest unit of software design**—the software component or module.
- Using the component-level design description as a guide, important control paths are tested to uncover errors within the boundary of the module. (Pressman, 2010)

© Fakultas Ilmu Komputer, Universitas Brawijaya

Aspek-aspek pengujian White-Box

- memastikan bahwa **semua jalur algoritma** telah diuji minimal sekali
- menguji seluruh keputusan **lojik** (true atau false)
- mengeksekusi seluruh **loop** dalam batasan yang ditentukan
- memvalidasi **struktur data** internal

© Fakultas Ilmu Komputer, Universitas Brawijaya

Struktur Kontrol Program

Setiap kode, sekompleks apa pun, dibangun dari tiga struktur dasar logika:

1. **Sequence:** Eksekusi pernyataan secara berurutan.
2. **Selection (Percabangan):** Pengambilan keputusan berdasarkan kondisi (contoh: `if-else`, `switch-case`).
3. **Iteration (Perulangan):** Eksekusi blok kode berulang kali (contoh: `for`, `while`, `do-while`).

Di industri, bug seringkali bersembunyi di struktur selection dan batas kondisi pada iteration (off-by-one errors).

© Fakultas Ilmu Komputer, Universitas Brawijaya

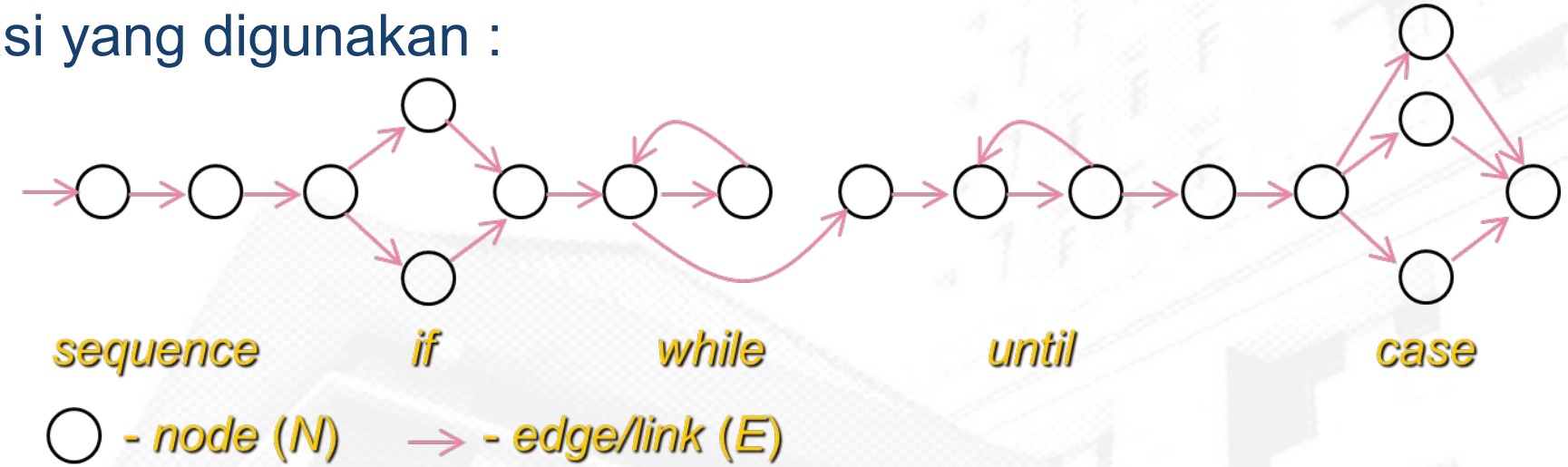
White Box Testing

- Jenis :
  - Pengujian Jalur Dasar (***Basis Path Testing***)
  - Pengujian Struktur Kontrol (***Control Structure Testing***)
- Pengujian Jalur Dasar (***Basis Path Testing***)
  - Pengujian kotak putih yang dibuat berdasarkan ukuran **tingkat kompleksitas** dari algoritma hasil perancangan
  - Langkah-langkah :
    i. Mendefinisikan flow graph berdasarkan mapping dari flow chart atau struktur algoritma
    ii. Menentukan ukuran kompleksitas (cyclomatic complexity)
    iii. Mendefinisikan kasus uji

© Fakultas Ilmu Komputer, Universitas Brawijaya

Basis Path Testing : Flow Graph

- **Notasi** sederhana yang merepresentasikan **aliran kontrol**
- Notasi yang digunakan :



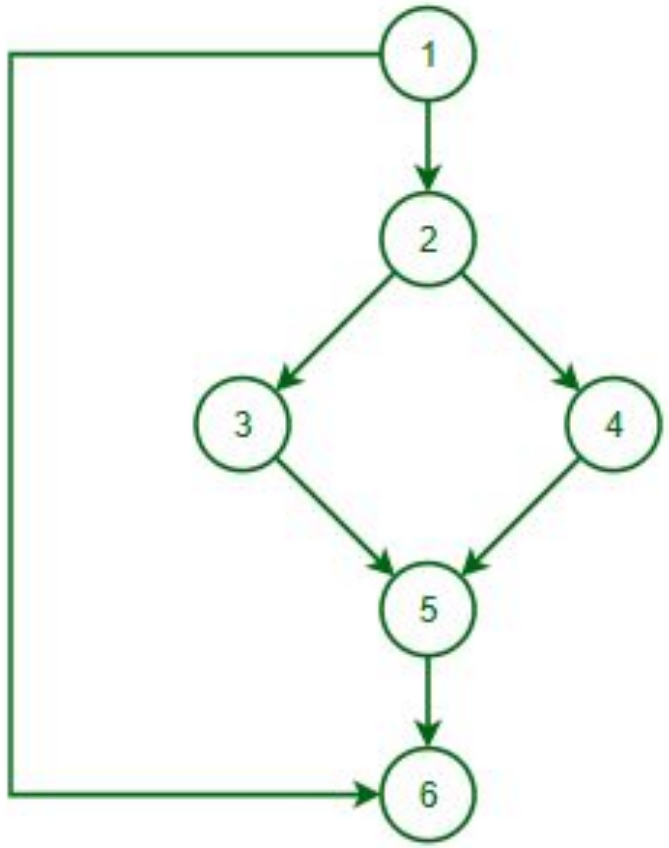
- proses dan keputusan yg berurutan dimapping menjadi 1 *node*
- setiap *edge* harus berakhir pada sebuah *node* (walaupun tidak merepresentasikan proses apapun)
- ***region*** : daerah yang dibatasi oleh *edge* dan *node*
- *predicate node* : *node* yg merupakan kondisi (2 atau lebih *edge* akan keluar dari sini)

Control Flow Graph (CFG)

- CFG adalah representasi grafis dari **semua jalur** yang mungkin **dilalui** oleh program selama eksekusi.
- Untuk menganalisis struktur kontrol secara matematis, kita mengubah kode program menjadi **Control Flow Graph (CFG)**.

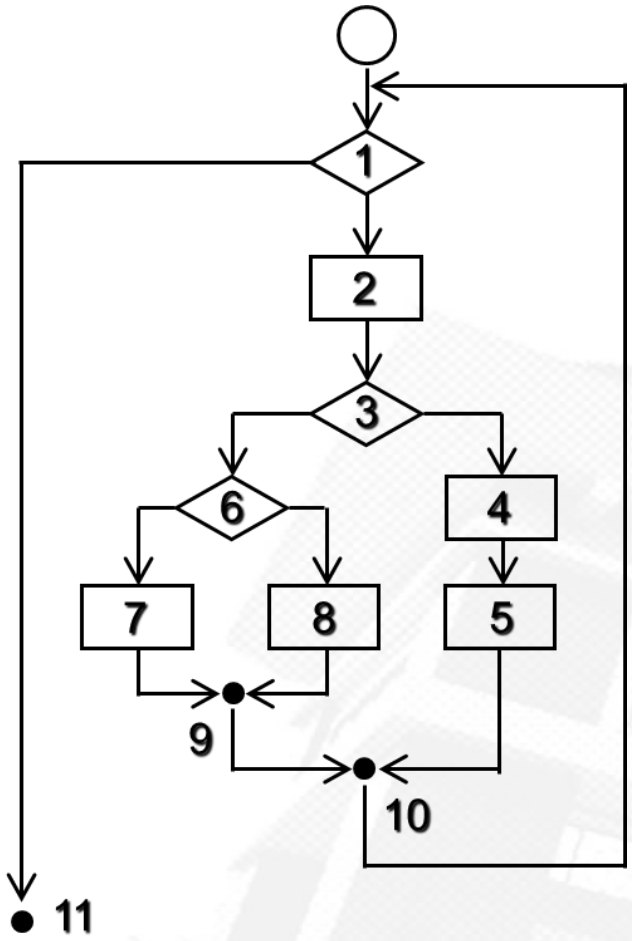
Komponen CFG:

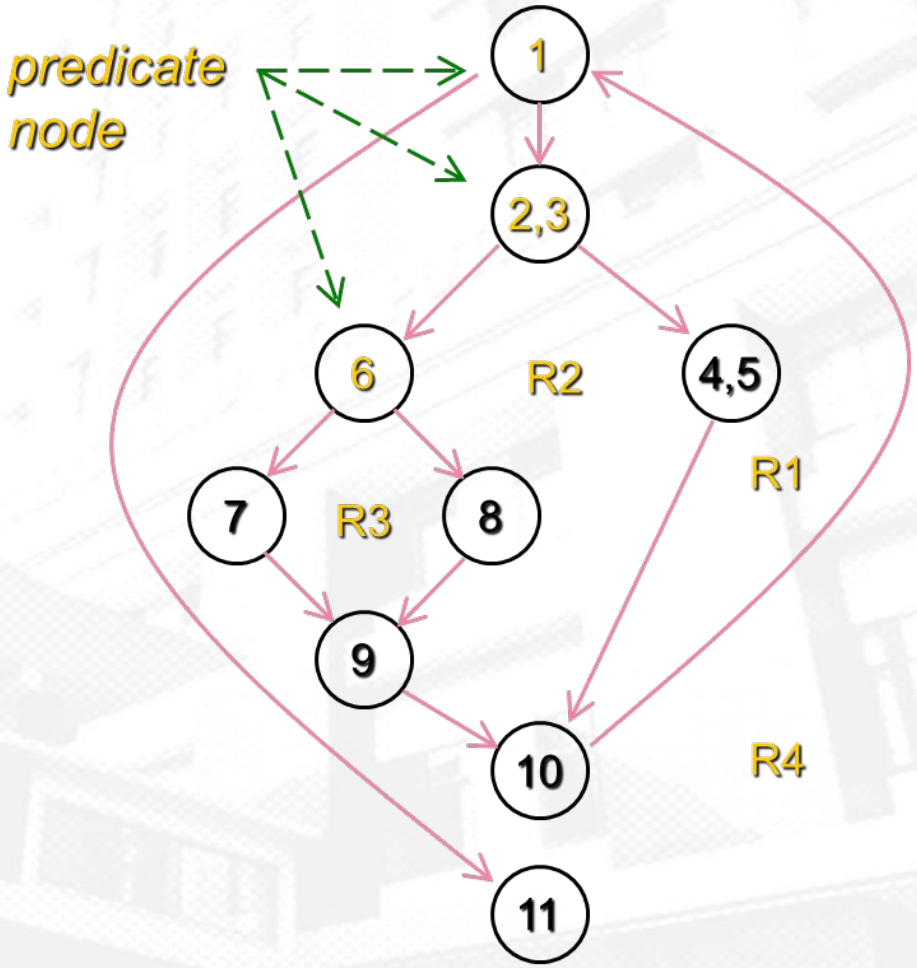
- **Node:** Merepresentasikan satu atau beberapa baris pernyataan prosedural berurutan (tanpa percabangan).
- **Predicate Node**: Simpul yang berisi kondisi (memiliki >1 panah keluar).
- **Edge (Panah):** Merepresentasikan aliran kontrol antar *Node*.
- **Entry Node:** Titik awal eksekusi modul.
- **Exit Node:** Titik akhir eksekusi modul.
- **Region**: daerah yang dibatasi oleh edge dan node



sumber: https://www.geeksforgeeks.org/software-engineering/software-engineering-control-flow-graph-cfg/

Transformasi dari flowchart ke CFG





Basis Path Testing : Cyclomatic Complexity

- **Definisi** : angka yang menyatakan jumlah jalur independen/jalur dasar dari sebuah program (representasi dari kompleksitas program)
- Menunjukkan jumlah pengujian (**kasus uji**) yang harus dieksekusi
- Jalur independen (*independent path*) : setiap jalur dalam program yang memiliki setidaknya **satu set pernyataan** (*processing statement*) atau satu kondisi yang baru sama sekali (belum digunakan oleh jalur sebelumnya)

© Fakultas Ilmu Komputer, Universitas Brawijaya

Coverage Criteria

Bagaimana kita tahu pengujian *white-box* kita sudah cukup? Kita menggunakan *Coverage Criteria*.

1. **Statement Coverage:** Metrik dasar. Memastikan setiap baris kode dieksekusi minimal satu kali.
   - *Kelemahan:* Bisa melewatkan bug pada kondisi boolean yang bernilai salah.
2. **Branch (Decision) Coverage:** Memastikan setiap cabang dari struktur kontrol (`if/else`, `case`) dieksekusi untuk nilai `True` dan `False`.
   - *Standar Industri:* Ini adalah metrik minimal yang sering diwajibkan (misalnya, di CI/CD pipeline).

© Fakultas Ilmu Komputer, Universitas Brawijaya

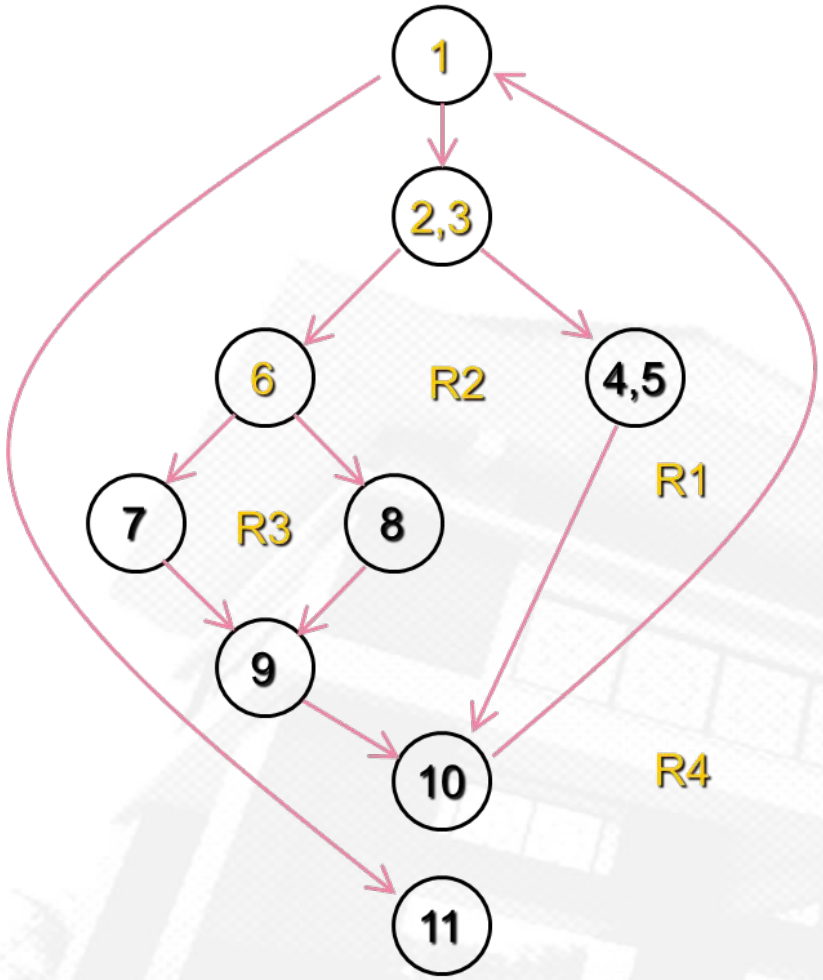
Coverage Criteria

3. **Condition Coverage:** Memastikan setiap kondisi boolean (sub-ekspresi) dalam sebuah *decision* dievaluasi ke `True` dan `False`.
4. **Multiple Condition Coverage:** Menguji semua kombinasi kemungkinan nilai kebenaran dari berbagai kondisi dasar dalam sebuah *decision*. Sangat ketat, namun eksponensial terhadap kompleksitas.
5. **Path Coverage (Konseptual):** Eksekusi semua rute perjalanan yang mungkin terjadi dalam program dari *Entry* ke *Exit*.
   - *Kenyataan:* Nyaris mustahil (infeasible) dipraktikkan jika kode memiliki perulangan (*loops*), karena jumlah jalurnya bisa mendekati tak terhingga.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Basis Path Testing : Cyclomatic Complexity

contoh:



- Jalur independen :
  - jalur 1 : 1-11
  - jalur 2 : 1-2-3-4-5-10-1-11
  - jalur 3 : 1-2-3-6-8-9-10-1-11
  - jalur 4 : 1-2-3-6-7-9-10-1-11

- Bukan Jalur independen :
  - 1-2-3-4-5-10-1-2-3-6-8-9-10-1-11

Basis Path Testing : Cyclomatic Complexity

Metrik perangkat lunak yang dikembangkan oleh Thomas McCabe (1976) **untuk mengukur kompleksitas logis suatu program**. Metrik ini mendefinisikan batas atas jumlah pengujian yang harus dilakukan.

Rumus:

$$V(G) = E - N + 2P$$

Keterangan:

- ***E*** = Jumlah *Edge*
- ***N*** = Jumlah *Node*
- ***P*** = Jumlah komponen yang terhubung (biasanya 1 untuk program utama tunggal)

Aturan Praktis Industri: Jika V(G) > 10, modul tersebut terlalu kompleks dan memiliki risiko cacat yang tinggi. Pertimbangkan untuk melakukan *refactoring*.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Basis Path Testing : Cyclomatic Complexity

Perhitungan matematis cyclomatic complexity – V(G) :

- ***V(G) = jumlah region***
- ***V(G) = E – N + 2***
- ***V(G) = P + 1, dimana P – predicate node***

Contoh perhitungan :

- V(G) = 4, ada 4 region R1, R2, R3, R4
- V(G) = 11 edges – 9 nodes + 2 = 4
- V(G) = 3 predicate node + 1 = 4

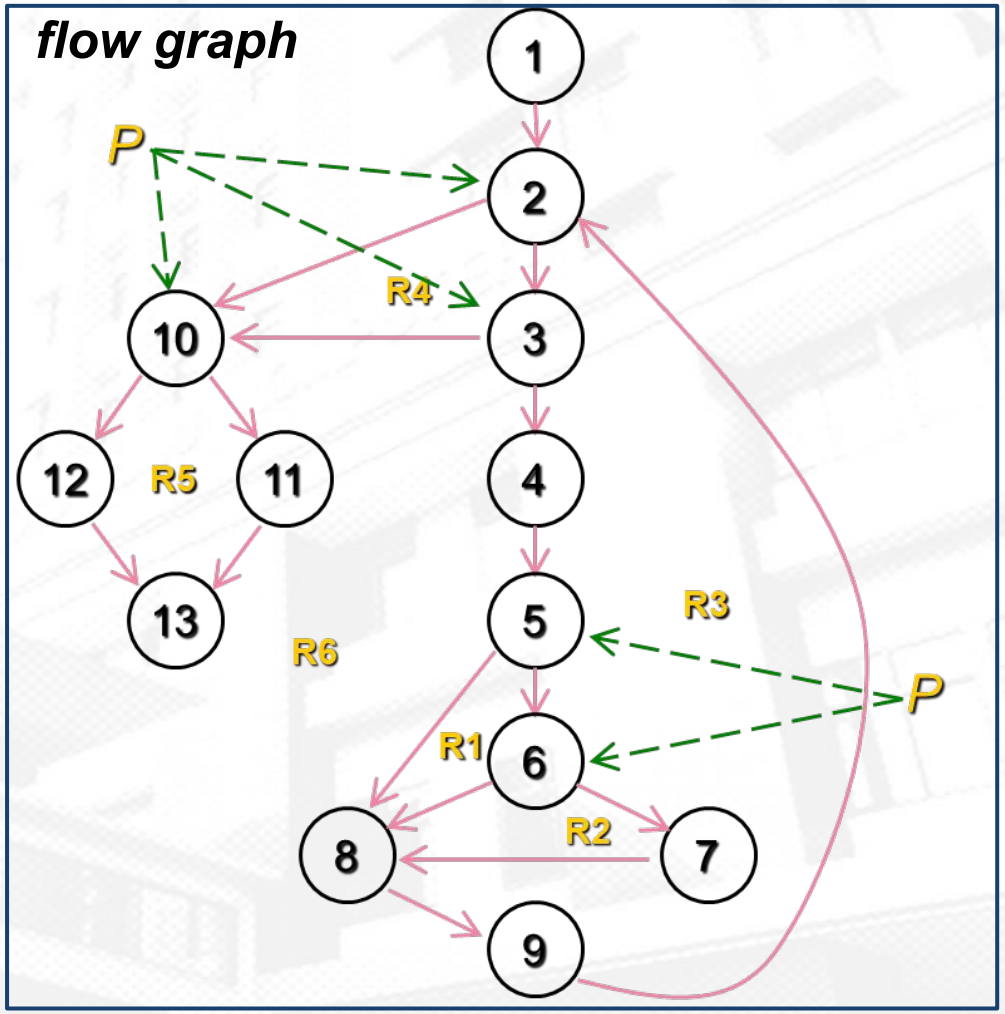
© Fakultas Ilmu Komputer, Universitas Brawijaya

Basis Path Testing : Contoh

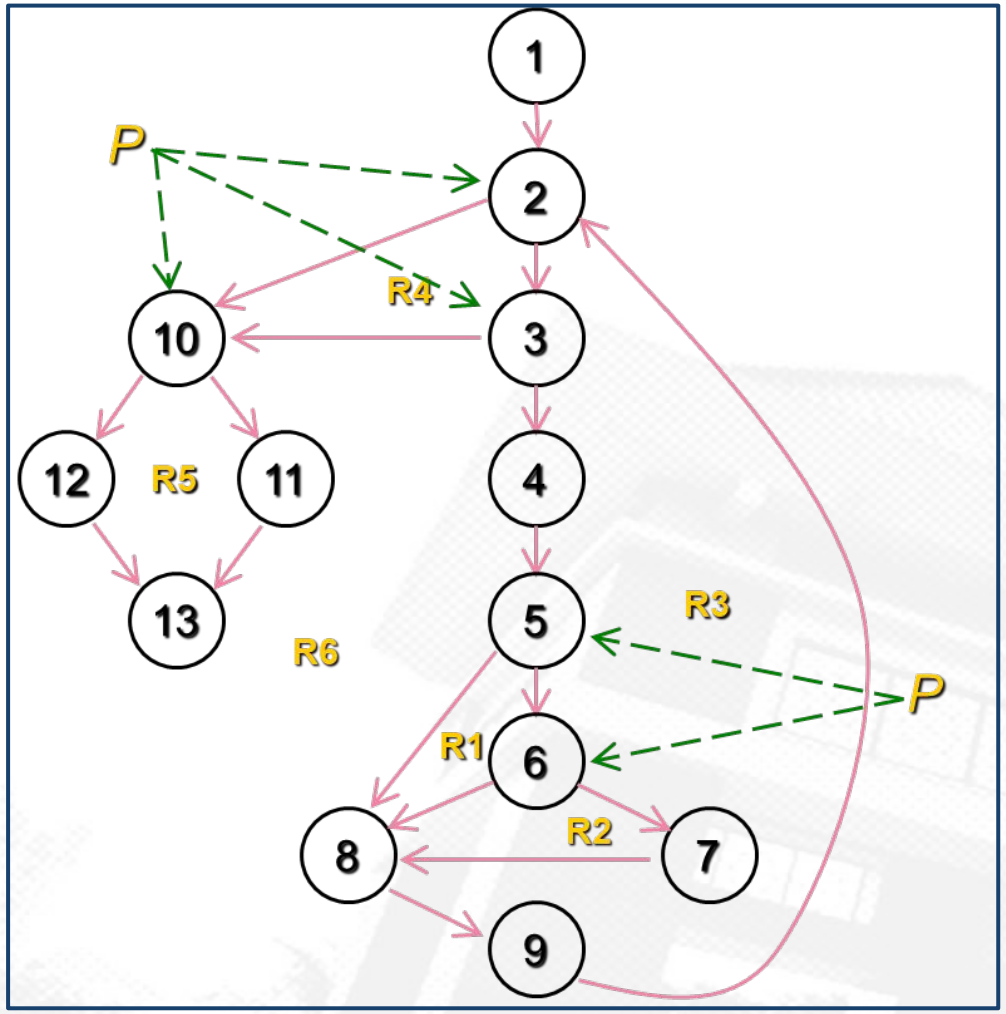
```
PROCEDURE average;
        INTERFACE RETURNS average, total.input, total.valid;
        INTERFACE ACCEPTS value, minimum, maximum;
        TYPE        value[1:100] IS SCALAR ARRAY;
        TYPE        average, total.input, total.valid, minimum,
                    maximum, sum IS SCALAR;
        TYPE i IS INTEGER;
        i = 1;
        total.input = total.valid = sum = 0;
        DO WHILE value[i] <> -999 AND total.input < 100
                    increment total.input by 1;
                    IF value[i] >= minimum AND value[i] <= maximum
                                THEN        increment total.valid by 1;
                                            sum = sum + value[i];
                                ELSE        skip;
                    ENDIF
                    increment i by 1;
        ENDDO
        IF total.valid > 0
                    THEN average = sum / total.valid;
                    ELSE average = -999;
        ENDIF
END average
```

Basis Path Testing : Contoh

```
PROCEDURE average;
    INTERFACE RETURNS average, total.input, total.valid;
    INTERFACE ACCEPTS value, minimum, maximum;
    TYPE      value[1:100] IS SCALAR ARRAY;
    TYPE      average, total.input, total.valid, minimum,
              maximum, sum IS SCALAR;
    TYPE i IS INTEGER;
    i = 1;
    total.input = total.valid = sum = 0;
    DO WHILE value[i] <> -999 AND total.input < 100
        increment total.input by 1;
        IF value[i] >= minimum AND value[i] <= maximum
                THEN   increment total.valid by 1;
                       sum = sum + value[i];
                ELSE   skip;
        ENDIF
        increment i by 1;
    ENDDO
    IF total.valid > 0
        THEN average = sum / total.valid;
        ELSE average = -999;
    ENDIF
END average
```



Basis Path Testing : Contoh



- *Cyclomatic complexity* :
  - V(G) = 6 regions
  - V(G) = 17E – 13N + 2 = 6
  - V(G) = 5P + 1 = 6

- Jalur independen :
  - jalur 1 : 1-2-10-11-13
  - jalur 2 : 1-2-10-12-13
  - jalur 3 : 1-2-3-10-11-13
  - jalur 4 : 1-2-3-4-5-8-9-2-…
  - jalur 5 : 1-2-3-4-5-6-8-9-2-…
  - jalur 6 : 1-2-3-4-5-6-7-8-9-2-…

Basis Path Testing : Contoh

test case

Test Case	Jalur Independen (Path)	Syarat Logika Jalur	Contoh Data Input (Data Uji)	Hasil yang Diharapkan
TC1	**1-2(F)-10-11-13**	**Infeasible Path**: Secara logika sulit dicapai karena jika loop tidak dimasuki, total.valid tetap 0, sehingga tidak mungkin ke simpul 11.	- (Diabaikan dalam tes fungsional)	-
TC2	**1-2(F)-10-12-13**	Loop langsung berhenti di pengecekan pertama (Simpul 2 = False).	value[1] = -999	average = -999
TC3	**1-2(T)-3(F)-10-...**	Masuk ke pengecekan loop pertama tapi gagal di pengecekan jumlah input (Array penuh).	Masukkan 100 angka valid, lalu cek kondisi berhenti.	average = [hasil rata-rata]
TC4	**...-5(F)-8-9-...**	Masuk loop, tapi nilai **di bawah minimum** sehingga lari ke simpul 8 (Skip).	value[1] = 5, min = 10, max = 100, value[2] = -999	average = -999 (karena tidak ada yang valid)
TC5	**...-5(T)-6(F)-8-...**	Masuk loop, nilai di atas minimum tapi **di atas maximum** (Skip).	value[1] = 150, min = 10, max = 100, value[2] = -999	average = -999
TC6	**...-5(T)-6(T)-7-...**	Masuk loop, nilai **berada di dalam range** (Proses hitung di simpul 7).	value[1] = 50, min = 10, max = 100, value[2] = -999	average = 50.0

Cara membuat data uji

1. **Fokus pada Predicate Node (Simpul Keputusan):** Lihat simpul yang memiliki panah bercabang (2, 3, 5, 6, 10). Tanyakan: *"Nilai berapa yang membuat kondisi ini bernilai True? Dan nilai berapa yang membuat False?"*.
2. **Menangani Loop (Simpul 2 & 3):**
   - Untuk langsung keluar loop, berikan nilai pembatal (`-999`) pada indeks pertama.
   - Untuk menguji batas array (simpul 3), Anda harus mensimulasikan input hingga 100 kali.
3. **Menangani Kondisi Majemuk (Simpul 5 & 6):** Karena `IF` dipisahkan menjadi dua node (5 dan 6), Anda harus membuat tes yang gagal di langkah pertama (simpul 5) dan tes yang lolos simpul 5 tapi gagal di simpul 6.
4. Cukup ingat bahwa setiap nomor jalur mewakili satu skenario unik.
   - Jika jalurnya melewati simpul 8, maka butuh data yang salah/tidak valid.
   - Jika jalurnya melewati simpul 7, maka butuh data yang benar/valid.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Control Structure Testing

Pengujian struktur kontrol, sebagai pelengkap bagi pengujian jalur dasar (*basis path testing*)

- Jenis:
  - Pengujian Kondisi (*Condition Testing*)
  - Pengujian Loop (*Loop Testing*)
- Pengujian Kondisi (*Condition Testing*)
  - Pengujian kotak putih yang dibuat untuk menguji kondisi lojik dalam sebuah program
  - Jenis-jenis kondisi :
    - Kondisi Sederhana (*simple condition*)
    - Kondisi Majemuk (*compound condition*)

© Fakultas Ilmu Komputer, Universitas Brawijaya

Apa itu Pengujian Kondisi (Condition Testing)?

Pengujian Kondisi adalah teknik pengujian kotak putih (*white-box*) yang dirancang khusus untuk menguji kondisi logis yang terdapat dalam suatu modul program. Fokus utama dari teknik ini adalah memastikan bahwa setiap elemen logika dalam sebuah keputusan (*decision*) dievaluasi secara benar, baik untuk nilai benar (*True*) maupun salah (*False*).

Tujuannya adalah untuk mendeteksi kesalahan dalam ekspresi logika, seperti:

- Operator relasional yang salah (misal: menggunakan `>` padahal seharusnya `>=`).
- Operator logika yang salah (misal: menggunakan `AND` padahal seharusnya `OR`).
- Kesalahan penulisan variabel dalam kondisi.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Jenis-Jenis Kondisi

Dalam pengujian ini, kita membedakan kondisi menjadi dua kategori utama:

A. Kondisi Sederhana (Simple Condition)

Kondisi sederhana adalah variabel Boolean atau ekspresi relasional yang tidak dapat dipecah lagi menjadi logika yang lebih kecil.

- **Struktur:** Ekspresi relasi : E1 <rel-op> E2
- Di mana <rel-op> adalah operator relasional seperti: <, >, <=, >=, ==, atau !=.
- **Contoh:** nilai >= 60, isMember == true, atau jumlah < 100.

B. Kondisi Majemuk (Compound Condition)

Kondisi majemuk terdiri dari dua atau lebih kondisi sederhana yang digabungkan menggunakan operator logika Boolean.

- **Operator Logika:** AND (&&), OR (||), dan NOT (!).
- **Contoh:** (nilai >= 60) AND (kehadiran > 80%).
- Dalam pengujian formal, kondisi majemuk sering kali dipecah menjadi beberapa simpul (*nodes*) dalam *Control Flow Graph* (CFG) untuk memastikan setiap bagian dari kondisi majemuk tersebut diuji secara independen.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Condition Testing: Contoh

gunakan algoritma average yang telah kita bahas sebelumnya sebagai contoh nyata.

Skenario Kode (Pseudocode):

Perhatikan baris validasi nilai berikut:

Plaintext

```
IF value[i] >= minimum AND value[i] <= maximum THEN
  ...
```

Langkah Analisis Pengujian Kondisi:

1. **Identifikasi Kondisi Sederhana:**
   - C_1: value[i] >= minimum
   - C_2: value[i] <= maximum
2. **Identifikasi Kondisi Majemuk:**
   - Cmajemuk: C_1 **AND** C_2
3. **Membuat Data Uji (Running Example):**
   Untuk menguji kondisi majemuk ini secara menyeluruh, kita harus menyiapkan skenario yang mencakup semua kemungkinan hasil logika

Test ID	Input value[i]	Kondisi C1	Kondisi C2	Hasil Majemuk (C1 ∧ C2)	Aksi Program
TC-01	50 (Min: 10, Max: 100)	**True**	**True**	**True**	Masuk ke blok THEN (Update Sum)
TC-02	5 (Min: 10, Max: 100)	**False**	**True**	**False**	Masuk ke blok ELSE (Skip)
TC-03	150 (Min: 10, Max: 100)	**True**	**False**	**False**	Masuk ke blok ELSE (Skip)
TC-04	-5 (Min: 10, Max: 100)	**False**	**True**	**False**	Masuk ke blok ELSE (Skip)

Mengapa Pengujian Kondisi Sangat Penting?

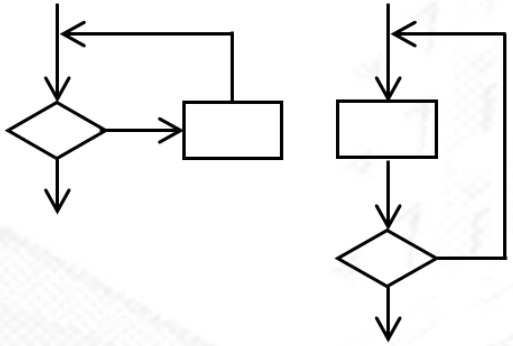
- Di industri, kesalahan paling umum terjadi pada batas kondisi (*boundary conditions*). Tanpa pengujian kondisi yang mendalam, kode Anda mungkin terlihat berjalan lancar, namun gagal ketika menghadapi nilai yang tepat berada di ambang batas (misalnya, program menerima nilai 10 ketika seharusnya hanya menerima nilai > 10).

- Dengan memetakan kondisi sederhana ke dalam test case seperti tabel di atas, Anda menjamin cakupan logika (*logic coverage*) yang jauh lebih tinggi daripada sekadar pengujian fungsional biasa.

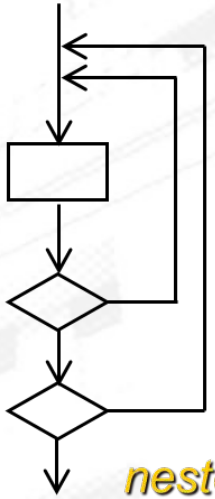
© Fakultas Ilmu Komputer, Universitas Brawijaya

Pengujian Loop (Loop Testing)

- Pengujian kotak putih yang dilakukan untuk menguji validitas dari struktur loop
- Jenis-jenis loop :



simple loop



nested loop

- Simple loop : dg. maks. n kali
  - Kasus uji tidak melewati loop sama sekali
  - Kasus uji m kali melewati loop, dimana m < n
  - Kasus uji n-1, n, n+1 melewati loop

© Fakultas Ilmu Komputer, Universitas Brawijaya

Pengujian Loop (Loop Testing)

- Nested loop : loop bersarang/loop di dalam loop, jumlah kasus uji semakin besar
- Pengujian dimulai dari loop paling dalam. Set iterator loop yang lain dengan nilai minimum
- Lakukan pengujian simple loop untuk loop paling dalam, sementara loop luarnya diset pada iterator yang minimum
- Contoh :

```
i, j: integer
DO WHILE i < 100
   j = 0
   DO WHILE j < 10
         Tampilkan nilai j ke layar
         Naikkan nilai j dengan 1
   ENDDO
   Naikkan nilai i dengan 1
ENDDO
```

Langkah pengujiannya adalah:

1. **Uji Loop Dalam secara penuh:** Sambil menahan `i` tetap di iterasi pertama, kita melakukan pengujian *simple loop* pada variabel `j` (misalnya menguji `j=0`, `j=1`, `j=9`, dan `j=10`).
2. **Set variabel `i` pada nilai minimum:** Kita menetapkan `i = 0` (hanya satu kali iterasi untuk loop luar).

Code Coverage Tools

Bagaimana Tools Bekerja? (Konsep Instrumentation)

- **Instrumentation:** Proses menyisipkan kode tambahan (secara otomatis oleh *tools*) ke dalam *source code* atau *bytecode* kita sebelum dieksekusi oleh *test suite*.
- *Counter* akan mencatat "Node/Edge mana yang dilewati" saat pengujian berjalan.

Tools Populer:

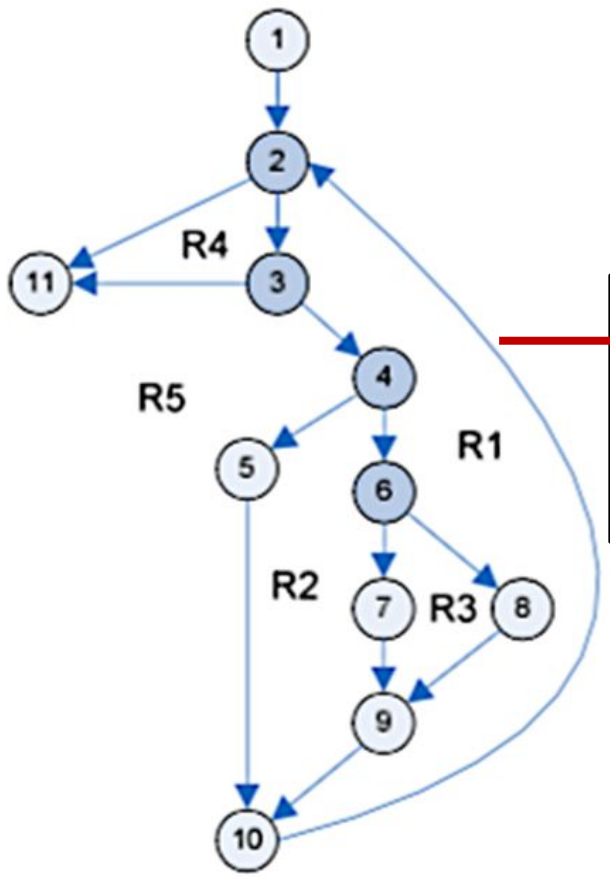
- **Java:** JaCoCo, Cobertura
- **JavaScript/TypeScript:** Istanbul (nyc), Jest Coverage
- **Python:** Coverage.py
- **C/C++:** Gcov

© Fakultas Ilmu Komputer, Universitas Brawijaya

```
PROCEDURE BINSEARCH (INPUT TABINT : INTEGER[],
X : INTEGER,N : INTEGER

OUTPUT IDX: INTEGER) {PROSEDUR MENCARI NILAI X DR SEBUAH ARRAY INTEGER}

{TABINT YANG TERURUT MENAIK, TABINT0 < TABINT1 <...} {< TABINTN, IDX AKAN BERNILAI -1 JIKA X TIDAK} {DITEMUKAN
ATAU BERNILAI POSISI INDEX DARI X YANG} {DITEMUKAN}

KAMUS:
KIRI, KANAN, TENGAH : INTEGER (INDEKS PENCARIAN) FOUND : BOOLEAN (TRUE JIKA DITEMUKAN)

ALGORITMA:
KIRI = 0
FOUND = FALSE
KANAN = N(PANJANG ARRAY TABINT) – 1

IDX = -1

WHILE (KIRI < KANAN) AND (NOT FOUND) DO

        TENGAH = (KIRI + KANAN)/2
        IF (X = TABINTTENGAH) THEN

                FOUND = TRUE

                IDX = TENGAH
        ELSE

        IF (X < TABINTTENGAH)

                KANAN = TENGAH – 1

                ELSE

                KIRI = TENGAH + 1

        ENDIF

ENDDO
```

Latihan (1/2)

```
Procedure BinSearch (Input  tabInt : integer[],
                            X : integer,n : integer
                     Output idX: integer)
{Prosedur mencari nilai X dr sebuah array integer}
{tabInt yang terurut menaik, tabInt0 ≤ tabInt1 ≤...}
{≤ tabIntn, idX akan bernilai -1 jika X tidak}
{ditemukan atau bernilai posisi index dari X yang}
{ditemukan}

Kamus:
kiri, kanan, tengah : integer (indeks pencarian)
found : boolean (TRUE jika ditemukan)

Algoritma:
kiri = 0                                  (2)   (3)
found = FALSE
kanan = n(panjang array tabInt) - 1       (1)
idX = -1
WHILE (kiri ≤ kanan) AND (NOT found) DO
  tengah = (kiri + kanan)/2
  IF (X = tabIntTengah) THEN              (4)
    found = TRUE
    idX = tengah                          (5)
  ELSE IF (X < tabIntTengah)
    kanan = tengah - 1                    (6)
  ELSE
    kiri = tengah + 1                     (7)
  ENDIF
ENDDO                                     (8)
(9) (10)
(11)
```



- Jalur dasar 1: ***1-2-11***
- Jalur dasar 2: 1-2-***3***-11
- Jalur dasar 3: 1-2-3-***4-5-10***-2
- Jalur dasar 4: 1-2-3-4-***6-7-9***-10-2-...
- Jalur dasar 5: 1-2-3-4-6-***8***-9-10-2-...

Latihan (2/2)

TABEL 1. CONTOH DATA UJI PENCARIAN BINER

Jalur	tabInt	n	X	Exp. Result, idX
1	{}	0	12	-1
2	{2, 4, 7, 10, 11}	5	1	-1
3	{2, 4, 7, 10, 11}	5	7	2
4	{2, 4, 7, 10, 11}	5	4	1
5	{2, 4, 7, 10, 11}	5	10	3

© Fakultas Ilmu Komputer, Universitas Brawijaya

Referensi

- Ammann, P., & Offutt, J. (2016). Introduction to Software Testing (2nd ed.). Cambridge University Press.
- Aniche, M. (2022). Effective Software Testing: A Developer's Guide. Manning Publications.
- Graham, D., van Veenendaal, E., Evans, I., & Black, R. (2012). Foundations of Software Testing: ISTQB Certification (3rd ed.). Cengage Learning.
- Perry, W. E. (2006). Effective Methods for Software Testing (3rd ed.). Wiley Publishing, Inc.
- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

© Fakultas Ilmu Komputer, Universitas Brawijaya